



TROUBLESHOOTING REPORT

VIRTUAL CONTROL
VMWARE CLOUD FOUNDATION SOLUTIONS

VCF Operations for Logs Troubleshooting Report

Detailed troubleshooting documentation including service recovery, Cassandra repair, memory tuning, and bootstrap procedures.

SERVICE RECOVERY

CASSANDRA

MEMORY TUNING

BOOTSTRAP

VCF 9.0

VMware Cloud Foundation

VCF Operations for Logs — Troubleshooting, Redeployment & Integration Handbook

Prepared by: Virtual Control LLC **Date:** March 25, 2026 **Document Version:** 2.1 **Classification:** Internal — Lab Environment

Shell Translation Note: The bash blocks in this document are accompanied by Windows PowerShell 5.1 sibling blocks where applicable. For full translation rules (cert bypass, here-strings, REST patterns, line continuation, password quoting), see [../POWERSHELL-TRANSLATION-REFERENCE.md](#). Appliance-side commands (anything executed inside the Logs/Photon appliance over SSH) are not translated — run those in the appliance shell as-is.

Table of Contents

1. Executive Summary
2. Environment Reference
3. Prerequisites and Tools
 - 3.1 Verify Python is Installed
 - 3.2 Verify ovftool is Installed
 - 3.3 Verify Network Access to vCenter
4. How to Create a vCenter REST API Session
 - 4.1 What is a vCenter REST API Session
 - 4.2 Create the Session Script
 - 4.3 Run the Script
 - 4.4 Troubleshooting Session Creation
5. Problem Statement
6. Phase 1 — Discovery: Locate the Logs VM
 - 6.1 List All VMs in vCenter
 - 6.2 Get Guest Identity and IP Address
7. Phase 2 — Connectivity Test
 - 7.1 Test All Service Ports
 - 7.2 Result: All Ports Refused
8. Phase 3 — Remote Diagnostics via Guest Operations API
 - 8.1 What is the Guest Operations API and Why Use It
 - 8.2 The Callback Mechanism
 - 8.3 Create the Diagnostic Script
 - 8.4 Run the Diagnostic Script
 - 8.5 Diagnostic Results
9. Phase 4 — Root Cause Analysis
 - 9.1 Root Cause 1: VM Undersized
 - 9.2 Root Cause 2: Cassandra Crash Loop
 - 9.3 Root Cause 3: Missing Java Library

- 9.4 Root Cause 4: Stuck Upgrade State
- 9.5 Combined Verdict
- 10. Phase 5 — Attempted In-Place Repair
 - 10.1 Resize VM to 16 GB
 - 10.2 Result: Cassandra Still Crashes
- 11. Phase 6 — Delete the Broken VM
 - 11.1 Why Redeployment is the Only Option
 - 11.2 Create the Deletion Script
 - 11.3 Run the Deletion Script
- 12. Phase 7 — Deploy New OVA via ovftool
 - 12.1 Locate the OVA File
 - 12.2 Inspect OVA Properties
 - 12.3 Deployment Size Options
 - 12.4 Build the ovftool Command
 - 12.5 Run the ovftool Command
 - 12.6 Expected Output
 - 12.7 Troubleshooting ovftool Failures
- 13. Phase 8 — Post-Deploy: Resize and Power On
 - 13.1 Create the Configuration Script
 - 13.2 Run the Configuration Script
- 14. Phase 9 — Verify the New Appliance
- 15. Phase 10 — Initial Configuration Wizard
 - 15.1 Open the Web UI
 - 15.2 Wizard Step 1: Admin Password
 - 15.3 Wizard Step 2: License Key
 - 15.4 Wizard Step 3: General Configuration
 - 15.5 Wizard Step 4: CEIP
 - 15.6 Wizard Step 5: Time Configuration (NTP)
 - 15.7 Wizard Step 6: SMTP
 - 15.8 Finish the Wizard
- 16. Phase 11 — Integrate with VCF Operations
 - 16.1 Configure on the Logs Side
 - 16.2 Configure on the VCF Operations Side
 - 16.3 Verify the Integration
- 17. Summary of All Root Causes
- 18. Lessons Learned
- Appendix A — Complete ovftool Deployment Command
- Appendix B — VM Discovery Script
- Appendix C — Guest Operations Diagnostic Script
- Appendix D — VM Resize and Power-On Script
- Appendix E — VM Deletion Script
- Index

1. Executive Summary

Where commands run in this RCA. Recovery touches:

- **VCF Operations for Logs appliance shell (SSH to vcf-ops-logs root)** — `service-control`, `systemctl`, `cassandra-cli`, log review under `/var/log/loginsight/`. Most service-level diagnostics happen here.
- **VCF Operations UI (browser)** — `https://<vcf-ops-fqdn>/` for adapter / agent integration, dashboard verification.
- **VCF Operations for Logs UI (browser)** — `https://<vcf-ops-logs-fqdn>/` for ingestion verification.
- **vCenter UI** — VM-level operations (snapshot, power cycle of the appliance VM).

Replace any credential placeholder like `<your-password>` with values from your vault.

On March 24, 2026, the VCF Operations for Logs appliance (version 9.0.1.0) was found to be deployed but non-functional in the VCF 9 lab environment. The appliance was powered on but all services were down — the web UI was unreachable, no log ingestion ports were listening, and the appliance was not integrated with VCF Operations.

Through systematic remote diagnostics using the vCenter Guest Operations API, four root causes were identified:

1. **VM undersized** — deployed as Extra Small (2 vCPU / 8 GB RAM) instead of the required minimum (4 vCPU / 16 GB)
2. **Cassandra database crash loop** — insufficient memory caused Cassandra to fail repeatedly, preventing all services from starting
3. **Corrupted Java libraries** — missing `jnr-posix` JAR file (`NoClassDefFoundError: jnr/posix/POSIXHandler`)
4. **Stuck upgrade state** — `UpgradeService.sendContinueUpgradeFailedNotification` looping on every restart

An in-place repair was attempted (resize VM to 16 GB) but Cassandra still crashed due to the corrupted Java libraries and stuck upgrade state. The appliance was determined to be unrecoverable.

Resolution: The broken VM was deleted and a fresh OVA was deployed using `ovftool` with correct sizing (Small + RAM upgraded to 16 GB). The new appliance was deployed to IP `192.168.1.242` with all network settings pre-configured via OVF properties.

2. Environment Reference

VCF Components:

Role	IP Address	FQDN	VM ID
vCenter Server	—	vcenter.lab.local	vm-18
VCF Operations	192.168.1.77	—	vm-4015
VCF Operations for Logs (broken)	192.168.1.242	logs.lab.local	vm-69 (deleted)
VCF Operations for Logs (new)	192.168.1.242	logs.lab.local	vm-11016
SDDC Manager	192.168.1.241	—	vm-68
Fleet Management	192.168.1.78	—	vm-4014
Remote Collector	—	—	vm-4016
NSX Manager	—	—	vm-58

Network Configuration:

Setting	Value
Subnet	192.168.1.0/24
Gateway	192.168.1.1
DNS Server	192.168.1.230
DNS Domain	lab.local
NTP Server	192.168.1.230

Infrastructure:

Component	Detail
Datacenter	vcenter-dc01
Cluster	vcenter-cl01
Datastore	vcenter-cl01-ds-vsan01 (vSAN, 902 GB free)
Port Group	vcenter-cl01-vds01-pg-esx-mgmt (dvportgroup-22)
ESXi Hosts	esxi01-esxi04.lab.local (4 hosts, all CONNECTED)

Credentials Used:

System	Username	Password	Purpose
vCenter REST API	administrator@vsphere.local		API authentication for all scripts
Logs VM (root)	root		Guest OS access for diagnostics
ovftool URI	administrator@vsphere.local		OVA deployment target (URL-encoded)

Software Versions:

Component	Version
VCF Operations for Logs OVA	9.0.1.0.24960345
Embedded Cassandra	Apache Cassandra 4.1.7
Embedded Java	OpenJDK 11.0.26
ovftool (workstation)	5.0.0 (build-24927197)
Python (workstation)	3.14
Guest OS	VMware Photon OS (64-bit)

Workstation Information:

Setting	Value
Workstation IP	192.168.1.160
OS	Windows 11 Pro
Python Command	<code>py -3</code> (not <code>python3</code>)

Setting	Value
ovftool Path	C:\Program Files (x86)\VMware\VMware Workstation\OVFTool\ovftool.exe

3. Prerequisites and Tools

Before starting any troubleshooting, verify the following tools are available on your Windows workstation.

3.1 Verify Python is Installed

Step 1: Open a terminal on your Windows workstation.

- Press **Windows Key + R**, type `cmd`, and press **Enter** to open a Command Prompt
- Alternatively, open **Git Bash** or **Windows Terminal**

Step 2: Check that Python 3 is installed by running:

```
py -3 --version
```

Expected output:

```
Python 3.14.0
```

If you see "py is not recognized": Python is not installed. Download and install Python from <https://www.python.org/downloads/>. During installation, check **"Add Python to PATH"**.

3.2 Verify ovftool is Installed

Step 3: Check that ovftool is available:

```
"/c/Program Files (x86)/VMware/VMware Workstation/OVFTool/ovftool.exe" --version
```

Or in Command Prompt (not Git Bash):

```
"C:\Program Files (x86)\VMware\VMware Workstation\OVFTool\ovftool.exe" --version
```

Expected output:

```
VMware ovftool 5.0.0 (build-24927197)
```

If ovftool is not found: It is installed with VMware Workstation Pro. If you do not have Workstation, download ovftool separately from the Broadcom support portal.

3.3 Verify Network Access to vCenter

Step 4: Confirm you can reach vCenter from your workstation:

```
ping vcenter.lab.local
```

Expected output:

```
Reply from 192.168.1.x: bytes=32 time=1ms TTL=64
```

If ping fails, check your DNS settings and network connectivity before proceeding.

4. How to Create a vCenter REST API Session

4.1 What is a vCenter REST API Session

The vCenter REST API allows you to manage your vSphere environment programmatically — you can list VMs, change hardware settings, power on/off VMs, delete VMs, and execute commands inside guest operating systems, all without opening the vSphere Client.

Before making any API calls, you must **authenticate** by creating a **session**. This is done by sending your vCenter username and password to the session endpoint. vCenter returns a **session token** — a unique string that proves you are authenticated. You then include this token in all subsequent API calls.

Why use Python scripts? The scripts in this handbook use Python because it is pre-installed on most systems and can make HTTPS requests without additional software. Each script is self-contained and copy-paste ready.

4.2 Create the Session Script

Step 5: On your workstation, open a text editor (Notepad, VS Code, or any editor).

Step 6: Create a new file and save it as `E:\VCF-Depot\test_session.py` (or any location you prefer).

Step 7: Copy and paste the following **complete script** into the file:

```
import urllib.request
import json
import ssl
import base64

# --- SSL Configuration ---
# VCF lab environments use self-signed certificates.
# These two lines tell Python to accept self-signed certs.
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE

# --- Credentials ---
# Change these values to match YOUR environment.
vcenter_host = "vcenter.lab.local"
username = "administrator@vsphere.local"
password = "<your-password>"

# --- Build the Authentication Header ---
# The vCenter API uses HTTP Basic Authentication for session creation.
# Basic Auth requires the credentials in the format "username:password"
```

```

# encoded as Base64.
credentials = username + ":" + password
encoded = base64.b64encode(credentials.encode()).decode()

# --- Create the API Session ---
# Send a POST request to /api/session with the Basic Auth header.
# The response is a JSON string containing the session token.
url = "https://" + vcenter_host + "/api/session"
request = urllib.request.Request(
    url,
    data=b"", # POST requires a body (empty is fine)
    headers={
        "Authorization": "Basic " + encoded
    },
    method="POST"
)

print("Connecting to vCenter at " + vcenter_host + "...")
try:
    response = urllib.request.urlopen(request, context=ctx)
    session_token = json.loads(response.read())
    print("SUCCESS: vCenter session created.")
    print("Session token: " + session_token[:20] + "...")
    print()
    print("This token is used in all subsequent API calls by including")
    print("the header: vmware-api-session-id: <token>")
except urllib.error.HTTPError as e:
    error_body = e.read().decode()[:300]
    print("FAILED: HTTP Error " + str(e.code))
    print("Response: " + error_body)
    print()
    if e.code == 401:
        print("CAUSE: Wrong username or password.")
        print("FIX: Verify the username and password in this script.")
    elif e.code == 503:
        print("CAUSE: vCenter services are not ready.")
        print("FIX: Wait a few minutes and try again.")
except urllib.error.URLError as e:
    print("FAILED: Cannot connect to vCenter.")
    print("Error: " + str(e.reason))
    print()
    print("POSSIBLE CAUSES:")
    print("  1. vCenter is unreachable (check: ping vcenter.lab.local)")
    print("  2. DNS cannot resolve vcenter.lab.local")
    print("  3. A firewall is blocking port 443")

```

Step 8: Save the file.

4.3 Run the Script

Step 9: Open a terminal (Command Prompt or Git Bash) and run:

```
py -3 E:\VCF-Depot\test_session.py
```

Expected output (success):

```

Connecting to vCenter at vcenter.lab.local...
SUCCESS: vCenter session created.
Session token: 5c2b8e4a1f3d7b9c...

This token is used in all subsequent API calls by including
the header: vmware-api-session-id: <token>

```

4.4 Troubleshooting Session Creation

Error	Cause	Fix
HTTP Error 401: Unauthorized	Wrong username or password	Verify <code>administrator@vsphere.local</code> and the password. The password <code><your-password></code> contains two exclamation marks — make sure both are included.
HTTP Error 503: Service Unavailable	vCenter services are starting up	Wait 2-3 minutes and retry. This happens after vCenter reboots.
[Errno 11001] getaddrinfo failed	DNS cannot resolve <code>vcenter.lab.local</code>	Check your DNS settings. Try using the IP address directly instead.
[WinError 10060] connection timed out	Network cannot reach vCenter	Verify your workstation is on the same network. Run <code>ping vcenter.lab.local</code> .
SyntaxWarning: "\"!" is an invalid escape sequence	Python 3.14+ string warning	This is a warning, not an error. The script still works. Use string concatenation (as shown above) instead of f-strings with the password.

Important: All remaining scripts in this handbook use the same authentication pattern shown above. Once you confirm this test script works, all other scripts will work too (they all connect to the same vCenter with the same credentials).

5. Problem Statement

The VCF Operations for Logs appliance (`logs.lab.local` , 192.168.1.242) was deployed in vCenter but was not integrated with VCF Operations. The appliance appeared as powered on in the vSphere Client, but:

- The web UI at `https://192.168.1.242` was unreachable
- No API endpoints responded on ports 443 or 9543
- No syslog ingestion ports (514, 1514, 9000) were listening
- The appliance did not appear in VCF Operations as an integrated solution

The goal was to determine why the appliance was not functional and restore it to a working state.

6. Phase 1 — Discovery: Locate the Logs VM

6.1 List All VMs in vCenter

Step 10: Create a new file called `E:\VCF-Depot\find_logs_vm.py` with the following **complete script**:

```
import urllib.request
import json
import ssl
import base64

# SSL setup (accept self-signed certs)
ctx = ssl.create_default_context()
ctx.check_hostname = False
```

```

ctx.verify_mode = ssl.CERT_NONE

# Authenticate to vCenter
password = "<your-password>"
credentials = "administrator@vsphere.local:" + password
encoded = base64.b64encode(credentials.encode()).decode()

request = urllib.request.Request(
    "https://vcenter.lab.local/api/session",
    data=b"",
    headers={"Authorization": "Basic " + encoded},
    method="POST"
)
response = urllib.request.urlopen(request, context=ctx)
session = json.loads(response.read())
print("vCenter session OK")

# List all VMs
request2 = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm",
    headers={"vmware-api-session-id": session}
)
response2 = urllib.request.urlopen(request2, context=ctx)
vms = json.loads(response2.read())

print("\nTotal VMs in inventory: " + str(len(vms)))
print("\nAll VMs:")
print("-" * 60)
for vm in sorted(vms, key=lambda x: x.get("name", "")):
    print(" " + vm["vm"] + " | " + vm["name"]
          + " | power=" + vm.get("power_state", "unknown"))

# Filter for logs-related VMs
print("\nVMs matching log/vrli/ops/insight keywords:")
print("-" * 60)
for vm in vms:
    name = vm.get("name", "").lower()
    if any(kw in name for kw in ["log", "vrli", "ops", "insight", "fleet"]):
        print(" " + vm["vm"] + " | " + vm["name"]
              + " | power=" + vm.get("power_state", "unknown"))

```

Step 11: Open a terminal and run:

```
py -3 E:\VCF-Depot\find_logs_vm.py
```

Output:

```

vCenter session OK

Total VMs in inventory: 10

All VMs:
-----
vm-4016 | collector | power=POWERED_ON
vm-4014 | fleet | power=POWERED_ON
vm-69 | logs | power=POWERED_ON
vm-58 | nsx-manager | power=POWERED_ON
vm-68 | sddc-manager | power=POWERED_ON
vm-1009 | test | power=POWERED_OFF
vm-18 | vcenter | power=POWERED_ON
vm-4015 | vcf-ops | power=POWERED_ON
...

VMs matching log/vrli/ops/insight keywords:
-----
vm-4014 | fleet | power=POWERED_ON

```

```
vm-4015 | vcf-ops | power=POWERED_ON
vm-69 | logs | power=POWERED_ON
```

What this tells us:

Column	Meaning
vm-69	The internal vCenter identifier for this VM. Used in all API calls.
logs	The display name of the VM in the vSphere Client.
POWERED_ON	The VM is running at the hypervisor level.

Key Finding: The Logs appliance is `vm-69`, named `logs`, and it is `POWERED_ON`. The VM exists and is running — the problem is with the application inside it, not the VM itself.

6.2 Get Guest Identity and IP Address

Step 12: Add the following to the bottom of `find_logs_vm.py` (or create a separate script), then run it:

```
# Get guest identity for the logs VM
print("\nGuest Identity for vm-69:")
print("-" * 60)
request3 = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm/vm-69/guest/identity",
    headers={"vmware-api-session-id": session}
)
response3 = urllib.request.urlopen(request3, context=ctx)
identity = json.loads(response3.read())
print(" Guest OS: " + str(identity.get("full_name")))
print(" Hostname: " + str(identity.get("host_name")))
print(" IP Address:" + str(identity.get("ip_address")))
print(" OS Family: " + str(identity.get("family")))
```

Output:

```
Guest Identity for vm-69:
-----
Guest OS:  VMware Photon OS (64-bit)
Hostname:  logs.lab.local
IP Address:192.168.1.242
OS Family: LINUX
```

What this tells us: VMware Tools is running inside the guest (that is how vCenter knows the IP and hostname). The VM has the correct IP (`192.168.1.242`) and hostname (`logs.lab.local`).

7. Phase 2 — Connectivity Test

7.1 Test All Service Ports

Step 13: Create a file called `E:\VCF-Depot\test_connectivity.py` with this **complete script**:

```

import socket
import ssl
import urllib.request

print("Testing connectivity to 192.168.1.242...")
print("=" * 60)

# Test 1: TCP port connectivity
ports = {
    22: "SSH (remote shell access)",
    80: "HTTP (web UI redirect)",
    443: "HTTPS (web UI and REST API)",
    514: "Syslog (TCP)",
    1514: "Syslog (SSL)",
    9000: "CFAPI (Ingestion API)",
    9042: "Cassandra native transport",
    9543: "CFAPI over SSL (Ingestion API)",
}

for port, description in sorted(ports.items()):
    try:
        sock = socket.create_connection(("192.168.1.242", port), timeout=5)
        sock.close()
        print(" Port " + str(port) + " (" + description + "): OPEN")
    except socket.timeout:
        print(" Port " + str(port) + " (" + description + "): TIMEOUT")
    except ConnectionRefusedError:
        print(" Port " + str(port) + " (" + description + "): REFUSED")
    except Exception as e:
        print(" Port " + str(port) + " (" + description + "): " + str(e))

# Test 2: HTTPS request to web UI
print()
print("Testing HTTPS request to web UI...")
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE
try:
    req = urllib.request.Request("https://192.168.1.242/")
    resp = urllib.request.urlopen(req, context=ctx, timeout=10)
    print(" Status: " + str(resp.status))
except Exception as e:
    print(" Result: " + str(e))

```

Step 14: Run it:

```
py -3 E:\VCF-Depot\test_connectivity.py
```

7.2 Result: All Ports Refused

Output:

```

Testing connectivity to 192.168.1.242...
=====
Port 22  (SSH):      OPEN
Port 80  (HTTP):     REFUSED
Port 443 (HTTPS):    REFUSED
Port 514 (Syslog TCP): REFUSED
Port 1514 (Syslog SSL): REFUSED
Port 9000 (CFAPI):   REFUSED
Port 9042 (Cassandra): REFUSED
Port 9543 (CFAPI over SSL): REFUSED

```

Testing HTTPS request to web UI...
Result: Connection refused

Problem Confirmed: Only SSH (port 22) is listening. All VCF Operations for Logs service ports (80, 443, 514, 9000, 9042, 9543) are refusing connections. The VM is running at the OS level (VMware Tools and SSH are active) but the Logs application is completely down.

8. Phase 3 — Remote Diagnostics via Guest Operations API

8.1 What is the Guest Operations API and Why Use It

Since the web UI and API are down, we cannot diagnose the appliance through its normal interfaces. SSH was available (port 22 was open), but the vCenter **Guest Operations API** provides an alternative way to run commands inside the VM without needing SSH access.

How it works:

1. You send an API request to vCenter (not to the Logs VM directly)
2. vCenter communicates with VMware Tools inside the guest VM
3. VMware Tools executes the command as the specified user (root)
4. The command output is captured and returned

API Endpoint:

```
POST https://vcenter.lab.local/api/vcenter/vm/{vm-id}/guest/processes?action=create
```

Required JSON body:

```
{
  "credentials": {
    "interactive_session": false,
    "type": "USERNAME_PASSWORD",
    "user_name": "root",
    "password": "<your-password>"
  },
  "spec": {
    "path": "/bin/bash",
    "arguments": "-c \"<your shell command here>\""
  }
}
```

Field-by-field explanation:

Field	Value	Why
interactive_session	false	We are running a batch command, not an interactive terminal
type	USERNAME_PASSWORD	This exact string is required by the API — it means "authenticate with a username and password"

Field	Value	Why
user_name	root	The Linux root user on the Logs appliance
password	<your-password>	The root password set during OVA deployment
path	/bin/bash	The shell binary to use for executing the command
arguments	-c "<command>"	The -c flag tells bash to run the quoted string as a command

8.2 The Callback Mechanism

The Guest Operations API creates a process inside the VM but does **not** return its stdout/stderr output. To capture the output, we use a **callback mechanism**:

1. **Start a temporary HTTP listener** on your workstation (e.g., port 9960)
2. **Execute the command** inside the guest VM, piping output to `curl`
3. `curl` sends the output as an HTTP POST back to your workstation
4. **Your listener captures the POST body** (which is the command output)

Example: To run `ss -tlnp` inside the VM and get the output back:

```
# The command sent to the guest VM:
ss -tlnp | curl -sk -X POST -d @- http://192.168.1.160:9960/result

# What this does:
# ss -tlnp          - runs the "show listening ports" command
# |                - pipes the output to the next command
# curl -sk          - sends an HTTP request (-s=silent, -k=ignore SSL)
# -X POST           - use the POST method
# -d @-             - read the POST body from stdin (the piped output)
# http://192.168.1.160:9960/result - your workstation's temporary listener
```

8.3 Create the Diagnostic Script

Step 15: Create a file called `E:\VCF-Depot\diag_logs.py` with the following **complete script**:

```
import urllib.request
import json
import ssl
import base64
import http.server
import threading
import time

# --- SSL and Authentication ---
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE

password = "<your-password>"
credentials = "administrator@vsphere.local:" + password
encoded = base64.b64encode(credentials.encode()).decode()

request = urllib.request.Request(
    "https://vcenter.lab.local/api/session",
    data=b"",
    headers={"Authorization": "Basic " + encoded},
    method="POST"
)
response = urllib.request.urlopen(request, context=ctx)
```

```

session = json.loads(response.read())
headers = {
    "vmware-api-session-id": session,
    "Content-Type": "application/json"
}
print("vCenter session OK")

# --- Configuration ---
VM_ID = "vm-69" # The broken Logs VM
WORKSTATION_IP = "192.168.1.160" # YOUR workstation IP
ROOT_PASSWORD = password # Root password of the Logs VM

guest_creds = {
    "interactive_session": False,
    "type": "USERNAME_PASSWORD",
    "user_name": "root",
    "password": ROOT_PASSWORD
}

def run_remote_command(command, listen_port, timeout_seconds=25):
    """
    Execute a command inside the guest VM and capture the output.

    1. Starts an HTTP listener on listen_port
    2. Sends the command to the VM via Guest Operations API
    3. The command pipes output to curl, which POSTs it back to us
    4. Returns the captured output as a string
    """
    captured = []

    class OutputHandler(http.server.BaseHTTPRequestHandler):
        def do_POST(self):
            length = int(self.headers.get("Content-Length", 0))
            body = self.rfile.read(length)
            captured.append(body.decode("utf-8", errors="replace"))
            self.send_response(200)
            self.end_headers()
            self.wfile.write(b"ok")
        def log_message(self, *args):
            pass # Suppress log output

    server = http.server.HTTPServer(("0.0.0.0", listen_port), OutputHandler)
    listener = threading.Thread(target=server.handle_request, daemon=True)
    listener.start()

    # Build the full command: run the user command, pipe to curl
    full_command = (command + " | curl -sk -X POST -d @- "
        "http://" + WORKSTATION_IP + ":" + str(listen_port) + "/r")

    payload = json.dumps({
        "credentials": guest_creds,
        "spec": {
            "path": "/bin/bash",
            "arguments": '-c "' + full_command + '"'
        }
    }).encode()

    try:
        req = urllib.request.Request(
            "https://vcenter.lab.local/api/vcenter/vm/"
            + VM_ID + "/guest/processes?action=create",
            data=payload,
            headers=headers,
            method="POST"
        )
        resp = urllib.request.urlopen(req, context=ctx)
        pid = json.loads(resp.read())
        print(" Process started (PID: " + str(pid) + ")")
    except urllib.error.HTTPError as e:
        print(" ERROR: " + str(e.code) + " - ")

```

```

        + e.read().decode()[:200])
        server.server_close()
        return "(error starting process)"

    listener.join(timeout=timeout_seconds)
    server.server_close()
    return captured[0] if captured else "(no response within timeout)"

# =====
# TEST 1: What ports are listening?
# =====
print("\n" + "=" * 60)
print("TEST 1: Network Listeners (ss -tlnp)")
print("=" * 60)
result = run_remote_command("ss -tlnp 2>&1", 9960)
print(result)
time.sleep(1)

# =====
# TEST 2: What VMware services are running?
# =====
print("\n" + "=" * 60)
print("TEST 2: Running VMware/Loginsight Services")
print("=" * 60)
result = run_remote_command(
    "systemctl list-units --type=service --state=running 2>&1 "
    "| grep -i 'vmware\\|loginsight'",
    9961
)
print(result if result.strip() else " (no loginsight services running)")
time.sleep(1)

# =====
# TEST 3: Is the loginsight service failed?
# =====
print("\n" + "=" * 60)
print("TEST 3: Loginsight Service Status")
print("=" * 60)
result = run_remote_command(
    "systemctl status loginsight 2>&1 | head -10",
    9962
)
print(result)
time.sleep(1)

# =====
# TEST 4: Disk usage
# =====
print("\n" + "=" * 60)
print("TEST 4: Disk Usage (df -h)")
print("=" * 60)
result = run_remote_command("df -h 2>&1", 9963)
print(result)
time.sleep(1)

# =====
# TEST 5: CPU and memory
# =====
print("\n" + "=" * 60)
print("TEST 5: CPU Count and Memory")
print("=" * 60)
result = run_remote_command(
    "echo 'CPUs:' $(nproc); head -3 /proc/meminfo",
    9964
)
print(result)
time.sleep(1)

# =====

```



```
# TEST 6: Installed version
# =====
print("\n" + "=" * 60)
print("TEST 6: Installed RPM Packages")
print("=" * 60)
result = run_remote_command(
    "rpm -qa 2>&1 | grep -i 'loginsight\\|vrli\\|vmware'",
    9965
)
print(result)
time.sleep(1)

# =====
# TEST 7: Runtime log errors
# =====
print("\n" + "=" * 60)
print("TEST 7: Runtime Log Errors (last 30 lines)")
print("=" * 60)
result = run_remote_command(
    "tail -200 /storage/var/loginsight/runtime.log 2>&1 "
    "| grep -i 'ERROR\\|WARN\\|fail\\|cassandra' | tail -15",
    9966
)
print(result)

print("\n" + "=" * 60)
print("DIAGNOSTICS COMPLETE")
print("=" * 60)
```

8.4 Run the Diagnostic Script

Step 16: Run the script:

```
py -3 -X utf8 E:\VCF-Depot\diag_logs.py
```

Note: The `-X utf8` flag is needed because some command output contains Unicode characters that Windows Command Prompt cannot display by default. If you are using Git Bash, you may not need this flag.

8.5 Diagnostic Results

TEST 1 — Network Listeners:

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	10	127.0.0.1:25	0.0.0.0:*	sendmail
LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	sshd
LISTEN	0	4096	127.0.0.54:53	0.0.0.0:*	systemd-resolve
LISTEN	0	4096	127.0.0.53:53	0.0.0.0:*	systemd-resolve

What this means: Only SSH (22), sendmail (25), and the local DNS resolver (53) are listening. **No VCF Operations for Logs ports are active** — no 80, no 443, no 514, no 9000, no 9042, no 9543. The application is completely down.

TEST 2 — Running Services:

```
vmtoolsd.service    loaded active running    Service for virtual machines hosted on VMware
```

What this means: Only VMware Tools is running. The `loginsight.service` is not listed.

TEST 3 — Loginsight Service Status:

```
loginsight.service - VCF Operations for Logs
Active: inactive (dead)
```

What this means: The loginsight service exists but is not running. It is `inactive (dead)`.

TEST 4 — Disk Usage:

```
/dev/sda4          7.6G  2.5G  4.8G  34% /
/dev/mapper/data-var 20G   92M   19G   1% /storage/var
/dev/mapper/data-core 482G  5.2M  457G   1% /storage/core
```

What this means: `/storage/core` has only **5.2 MB used** out of 482 GB. This is where Cassandra stores all ingested log data. An empty partition means the appliance **was never successfully initialized** — no logs were ever ingested.

TEST 5 — CPU and Memory:

```
CPUs: 2
MemTotal: 8126604 kB (approximately 7.75 GB)
```

What this means: The VM has only 2 vCPUs and ~8 GB RAM. This is the **Extra Small** deployment size.

TEST 7 — Runtime Log Errors:

```
[2026-03-24 00:16:43] WARN Connection refused: /192.168.1.242:9042
[2026-03-24 00:16:43] WARN Connection refused: /192.168.1.242:9042
[2026-03-24 00:16:43] WARN Connection refused: /192.168.1.242:9042
[2026-03-24 00:16:43] INFO ActiveMQ stopped
java.lang.NoClassDefFoundError: jnr/posix/POSIXHandler
[ERROR] UpgradeService.sendContinueUpgradeFailedNotification
```

What this means: - Port 9042 is Cassandra's native transport port. The application tried to connect but Cassandra was not running. - `NoClassDefFoundError: jnr/posix/POSIXHandler` — a required Java library is missing or corrupted. - `UpgradeService` error — the application is stuck in an upgrade loop.

9. Phase 4 — Root Cause Analysis

9.1 Root Cause 1: VM Undersized

The VM was deployed with only **2 vCPU and 8 GB RAM** (Extra Small size).

VCF Operations for Logs 9.0 sizing requirements:

Size	vCPU	RAM	Hosts Supported	Notes
Extra Small	2	4 GB	20	Test/POC only — do NOT use
Small	4	8 GB	200	Minimum for standalone
Medium	8	16 GB	500	Recommended for clusters
Large	16	32 GB	1,500	Enterprise scale

The Loginsight Java process is configured with `-Xmx3968m` (4 GB heap). Cassandra dynamically calculates its heap as approximately 3 GB. Combined: **7 GB for Java + OS overhead exceeds 8 GB total RAM**. The processes compete for memory and Cassandra gets killed.

Root Cause 1: The VM was deployed with insufficient resources. With only 8 GB total RAM, the Loginsight daemon (4 GB) and Cassandra (3 GB) cannot run simultaneously.

9.2 Root Cause 2: Cassandra Crash Loop

The runtime log shows Cassandra starting briefly then crashing in a repeating pattern:

```
[WARN] Connection refused: /192.168.1.242:9042
[WARN] Connection refused: /192.168.1.242:9042
[INFO] No cassandra hosts available after 4037 ms wait
```

The watchdog process detects Cassandra is down, restarts everything, Cassandra starts for a few seconds, then crashes again. This infinite loop consumed **2 hours 49 minutes of CPU time** before the service finally gave up.

9.3 Root Cause 3: Missing Java Library

```
java.lang.NoClassDefFoundError: jnr/posix/POSIXHandler
```

The `jnr-posix` library is required by Cassandra for native POSIX operations (file system access, process management). Without it, Cassandra cannot initialize. This indicates a **corrupted installation** — likely caused by the initial deployment running out of memory during first boot.

9.4 Root Cause 4: Stuck Upgrade State

```
[ERROR] UpgradeService.sendContinueUpgradeFailedNotification
[WARN] Queue isn't running that is expected during shutdown
[ERROR] Could not add notification to queue
```

The application is stuck in a `continue-upgrade` thread that fires on every restart, tries to send a notification, fails because ActiveMQ is shutting down, and the cycle repeats.

9.5 Combined Verdict

Combined Root Cause: The appliance was deployed as Extra Small (2 vCPU / 8 GB). During first boot, insufficient memory caused Cassandra and the application to compete for resources, resulting in a corrupted initialization. The corruption manifested as missing Java libraries and a stuck upgrade state. Even after resizing the VM to 16 GB, the corrupted state persisted. **The appliance required a full redeployment.**

10. Phase 5 — Attempted In-Place Repair

Before deciding to redeploy, an attempt was made to fix the existing VM by increasing resources.

10.1 Resize VM to 16 GB

The following changes were made via the vCenter REST API (the VM was powered off first):

Change	API Call	Result
Power off	POST /api/vcenter/vm/vm-69/guest/power?action=shutdown	204 OK
RAM → 16 GB	PATCH /api/vcenter/vm/vm-69/hardware/memory with {"size_MiB": 16384}	204 OK
CPU → 4	PATCH /api/vcenter/vm/vm-69/hardware/cpu with {"count": 4}	204 OK
Enable memory hot-add	PATCH /api/vcenter/vm/vm-69/hardware/memory with {"hot_add_enabled": true}	204 OK
Enable CPU hot-add	PATCH /api/vcenter/vm/vm-69/hardware/cpu with {"hot_add_enabled": true}	204 OK
Power on	POST /api/vcenter/vm/vm-69/power?action=start	204 OK

After boot, the service startup was monitored every 30 seconds by checking ports inside the guest:

Time	Ports Observed	Status
+60s	22, 25, 53, 16520	Loginsight daemon starting
+90s	22, 25, 53, 8090, 16520	Admin tool starting
+120s	22, 25, 53, 7199, 8090, 16520	Cassandra JMX starting
+150s	22, 25, 53, 7000, 7001, 9042 , 7199, 8090, 16520	Cassandra UP
+180s	22, 25, 53, 8090	Cassandra crashed again

10.2 Result: Cassandra Still Crashes

Cassandra came up briefly (port 9042 appeared at +150s) but crashed again by +180s. Even with 16 GB RAM, the underlying corruption (`NoClassDefFoundError: jnr/posix/POSIXHandler`) and stuck upgrade state prevented stable operation. **The repair attempt failed. Redeployment is required.**

11. Phase 6 — Delete the Broken VM

11.1 Why Redeployment is the Only Option

Factor	Assessment
User data on appliance	None — /storage/core had 5.2 MB used (empty)
Cassandra database	Corrupted — missing JAR files, unstable startup
Upgrade state	Stuck — UpgradeService error loop on every boot

Factor	Assessment
Initial configuration	Never completed — the web UI wizard was never run
Time to repair vs redeploy	Manual JAR injection + Cassandra rebuild vs. 20-minute OVA deploy

Decision: Delete the broken VM and deploy a fresh OVA.

11.2 Create the Deletion Script

Step 17: Create a file called `E:\VCF-Depot\delete_logs_vm.py` with this **complete script**:

```
import urllib.request
import json
import ssl
import base64
import time

# SSL and authentication
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE

password = "<your-password>"
credentials = "administrator@vsphere.local:" + password
encoded = base64.b64encode(credentials.encode()).decode()

request = urllib.request.Request(
    "https://vcenter.lab.local/api/session",
    data=b"",
    headers={"Authorization": "Basic " + encoded},
    method="POST"
)
response = urllib.request.urlopen(request, context=ctx)
session = json.loads(response.read())
headers = {
    "vmware-api-session-id": session,
    "Content-Type": "application/json"
}
print("vCenter session OK")

VM_ID = "vm-69" # <<< CHANGE THIS to the VM ID of your broken Logs VM

# Step 1: Verify the VM exists
print("\nStep 1: Verify VM " + VM_ID + "...")
req = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm/" + VM_ID,
    headers=headers
)
resp = urllib.request.urlopen(req, context=ctx)
vm = json.loads(resp.read())
print(" Name: " + vm.get("name", "unknown"))
print(" Power: " + vm.get("power_state", "unknown"))

# Step 2: Power off if running
if vm["power_state"] == "POWERED_ON":
    print("\nStep 2: Powering off...")
    req2 = urllib.request.Request(
        "https://vcenter.lab.local/api/vcenter/vm/" + VM_ID
        + "/power?action=stop",
        data=b"", headers=headers, method="POST"
    )
    try:
        urllib.request.urlopen(req2, context=ctx)
        print(" Power off command sent. Waiting...")
    except urllib.error.HTTPError as e:
        print(" Error: " + str(e.code))
```

```

# Wait for power off
for i in range(12):
    time.sleep(5)
    req3 = urllib.request.Request(
        "https://vcenter.lab.local/api/vcenter/vm/" + VM_ID,
        headers=headers
    )
    resp3 = urllib.request.urlopen(req3, context=ctx)
    state = json.loads(resp3.read()).get("power_state")
    print(" [" + str(i * 5) + "s] Power: " + state)
    if state == "POWERED_OFF":
        break
else:
    print("\nStep 2: VM already powered off")

# Step 3: Delete the VM (removes from inventory AND deletes disk files)
print("\nStep 3: Deleting VM from disk...")
req4 = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm/" + VM_ID,
    headers=headers, method="DELETE"
)
try:
    resp4 = urllib.request.urlopen(req4, context=ctx)
    print(" Delete: " + str(resp4.status) + " - SUCCESS")
except urllib.error.HTTPError as e:
    print(" Delete error: " + str(e.code) + " - "
        + e.read().decode()[:300])

# Step 4: Verify deletion
print("\nStep 4: Verifying deletion...")
time.sleep(3)
try:
    req5 = urllib.request.Request(
        "https://vcenter.lab.local/api/vcenter/vm/" + VM_ID,
        headers=headers
    )
    urllib.request.urlopen(req5, context=ctx)
    print(" WARNING: VM still exists!")
except urllib.error.HTTPError as e:
    if e.code == 404:
        print(" CONFIRMED: VM deleted – no longer in inventory")
    else:
        print(" Unexpected error: " + str(e.code))

# Step 5: Show remaining VMs
print("\nRemaining VMs:")
req6 = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm",
    headers=headers
)
resp6 = urllib.request.urlopen(req6, context=ctx)
vms = json.loads(resp6.read())
for v in sorted(vms, key=lambda x: x.get("name", "")):
    print(" " + v["vm"] + " | " + v["name"]
        + " | " + v.get("power_state", ""))

```

11.3 Run the Deletion Script

Step 18: Run:

```
py -3 E:\VCF-Depot\delete_logs_vm.py
```

Expected output:

vCenter session OK

Step 1: Verify VM vm-69...

Name: logs

Power: POWERED_ON

Step 2: Powering off...

Power off command sent. Waiting...

[0s] Power: POWERED_OFF

Step 3: Deleting VM from disk...

Delete: 204 - SUCCESS

Step 4: Verifying deletion...

CONFIRMED: VM deleted – no longer in inventory

Remaining VMs:

vm-4016 | collector | POWERED_ON

vm-4014 | fleet | POWERED_ON

vm-58 | nsx-manager | POWERED_ON

vm-68 | sddc-manager | POWERED_ON

vm-18 | vcenter | POWERED_ON

vm-4015 | vcf-ops | POWERED_ON

VM deleted successfully. The broken logs VM (vm-69) has been removed from both the vCenter inventory and its disk files on the datastore.

12. Phase 7 — Deploy New OVA via ovftool

12.1 Locate the OVA File

The OVA file is located in the offline depot at:

```
E:\VCF-Depot\PROD\COMP\VR\LI\Operations-Logs-Appliance-9.0.1.0.24960345.ova
```

File size: 1,458 MB (1.42 GB)

12.2 Inspect OVA Properties

Step 19: Before deploying, inspect the OVA to see all configurable properties. Open a terminal and run:

```
"/c/Program Files (x86)/VMware/VMware Workstation/OVFTool/ovftool.exe" --hideEula "E:\VCF-Depot\PROD\COMP\VR\LI\Operations-Logs-Appliance-9.0.1.0.24960345.ova"
```

This shows all OVF properties that can be set at deployment time:

Property Key	Category	What to Set
rootpw	Application	Root password for the appliance
hostname	Application	FQDN (e.g., logs.lab.local)
preferipv6	Application	False for IPv4

Property Key	Category	What to Set
fips	Application	False unless FIPS mode is required
vami.ip0.VMware_vCenter_Log_Insight	Networking	IP address
vami.netmask0.VMware_vCenter_Log_Insight	Networking	Subnet mask
vami.gateway.VMware_vCenter_Log_Insight	Networking	Default gateway
vami.DNS.VMware_vCenter_Log_Insight	Networking	DNS server IP
vami.domain.VMware_vCenter_Log_Insight	Networking	Domain name
vami.searchpath.VMware_vCenter_Log_Insight	Networking	DNS search domain

12.3 Deployment Size Options

ID	Label	vCPU	RAM	Hosts Supported	Notes
xsmall	Extra Small	2	4 GB	20	Test/POC only — do NOT use
small	Small (default)	4	8 GB	200	We use this, then resize RAM to 16 GB after
medium	Medium	8	16 GB	500	May fail if hosts lack free resources
large	Large	16	32 GB	1,500	Enterprise scale

Why not deploy as Medium directly? In this environment, Medium (8 vCPU / 16 GB) failed with "No host is compatible with the virtual machine" because DRS was disabled and no single host had 16 GB free. The workaround is to deploy as **Small** (4 vCPU / 8 GB) then resize RAM to 16 GB post-deployment while the VM is still powered off.

12.4 Build the ovftool Command

Step 20: The complete ovftool command is below. **Before running it, review the parameter table and adjust any values for your environment.**

Parameter	Value in This Environment	Adjust?
--name	logs	Change if you want a different VM name
--deploymentOption	small	Use <code>medium</code> if hosts have enough resources
--datastore	vcenter-cl01-ds-vsan01	Change to your datastore name
--network	vcenter-cl01-vds01-pg-esx-mgmt	Change to your port group name
--prop:rootpw	<your-password>	Change to your desired root password
--prop:hostname	logs.lab.local	Change to your FQDN
--prop:vami.ip0	192.168.1.242	Change to your desired IP
--prop:vami.netmask0	255.255.255.0	Change to your subnet mask

Parameter	Value in This Environment	Adjust?
--prop:vami.gateway	192.168.1.1	Change to your gateway
--prop:vami.DNS	192.168.1.230	Change to your DNS server
Target URI	vcenter.lab.local/vcenter-dc01/host/vcenter-cl01/	Change to your vCenter/datacenter/cluster

Understanding the Target URI:

```
vi://administrator%40vsphere.local:<url-encoded-password>@vcenter.lab.local/vcenter-dc01/host/vcenter-cl01/
```

Part	Value	Explanation
vi://	Protocol prefix	Tells ovftool this is a vSphere target
administrator%40vsphere.local	Username	@ is URL-encoded as %40
<url-encoded-password>	Password	Each ! is URL-encoded as %21
@vcenter.lab.local	vCenter hostname	Separated from password by @
/vcenter-dc01	Datacenter name	Must match exactly
/host/vcenter-cl01/	Cluster path	Must include /host/ prefix

Flag-by-flag explanation:

Flag	Purpose
--acceptAllEulas	Automatically accept the VMware license agreements in the OVA
--skipManifestCheck	Skip the OVA checksum validation (faster deployment)
--noSSLVerify	Do not verify the vCenter SSL certificate (self-signed in lab)
--X:injectOvfEnv	Inject the OVF environment variables into the VM so the appliance reads them on first boot
--X:enableHiddenProperties	Allow setting properties that are marked as hidden in the OVF descriptor
--name="logs"	Set the VM display name in vCenter
--deploymentOption="small"	Select the Small deployment profile (4 vCPU / 8 GB)
--diskMode="thin"	Use thin provisioning to save datastore space

12.5 Run the ovftool Command

Step 21: Open a terminal (Git Bash recommended) and paste the following command **as a single command**:

```
# Workstation-side: deploy the Logs OVA to vCenter via ovftool with all vApp properties pre-set.
"/c/Program Files (x86)/VMware/VMware Workstation/OVFTool/ovftool.exe" \
--acceptAllEulas \
--skipManifestCheck \
```

```
--noSSLVerify \
--X:injectOvfEnv \
--X:enableHiddenProperties \
--name="logs" \
--deploymentOption="small" \
--diskMode="thin" \
--datastore="vcenter-cl01-ds-vsan01" \
--network="vcenter-cl01-vds01-pg-esx-mgmt" \
--prop:rootpw=<your-password> \
--prop:hostname="logs.lab.local" \
--prop:preferipv6="False" \
--prop:fips="False" \
--prop:vami.gateway.VMware_vCenter_Log_Insight="192.168.1.1" \
--prop:vami.domain.VMware_vCenter_Log_Insight="lab.local" \
--prop:vami.searchpath.VMware_vCenter_Log_Insight="lab.local" \
--prop:vami.DNS.VMware_vCenter_Log_Insight="192.168.1.230" \
--prop:vami.ip0.VMware_vCenter_Log_Insight="192.168.1.242" \
--prop:vami.netmask0.VMware_vCenter_Log_Insight="255.255.255.0" \
"E:/VCF-Depot/PROD/COMP/VRLI/Operations-Logs-Appliance-9.0.1.0.24960345.ova" \
"vi://administrator%40vsphere.local:<url-encoded-password>@vcenter.lab.local/vcenter-dc01/host/vcenter-cl01/"
```

```
# PowerShell 5.1 sibling – uses native Windows paths, backtick line continuation, and SINGLE-quoted
# strings around every value containing '!' or '$' so PowerShell does not expand them.
# No 'set +H' equivalent is needed in PowerShell – single-quoted strings are literal.
# URL-encode '@' as %40 inside the vi:// URI exactly as in the bash version.
```

```
& 'C:\Program Files (x86)\VMware\VMware Workstation\OVFTool\ovftool.exe' `
--acceptAllEulas `
--skipManifestCheck `
--noSSLVerify `
--X:injectOvfEnv `
--X:enableHiddenProperties `
--name=logs `
--deploymentOption=small `
--diskMode=thin `
'--datastore=vcenter-cl01-ds-vsan01' `
'--network=vcenter-cl01-vds01-pg-esx-mgmt' `
'--prop:rootpw=<your-password>' `
'--prop:hostname=logs.lab.local' `
'--prop:preferipv6=False' `
'--prop:fips=False' `
'--prop:vami.gateway.VMware_vCenter_Log_Insight=192.168.1.1' `
'--prop:vami.domain.VMware_vCenter_Log_Insight=lab.local' `
'--prop:vami.searchpath.VMware_vCenter_Log_Insight=lab.local' `
'--prop:vami.DNS.VMware_vCenter_Log_Insight=192.168.1.230' `
'--prop:vami.ip0.VMware_vCenter_Log_Insight=192.168.1.242' `
'--prop:vami.netmask0.VMware_vCenter_Log_Insight=255.255.255.0' `
'E:/VCF-Depot/PROD/COMP/VRLI/Operations-Logs-Appliance-9.0.1.0.24960345.ova' `
'vi://administrator%40vsphere.local:<url-encoded-password>@vcenter.lab.local/vcenter-dc01/host/vcenter-cl01/'
```

If using Windows Command Prompt instead of Git Bash: Replace all `\` line continuations with `^` and use Windows-style paths (`C:\` instead of `/c/`).

12.6 Expected Output

The deployment takes approximately 10-15 minutes depending on network speed:

```
Opening OVA source: E:/VCF-Depot/PROD/COMP/VRLI/Operations-Logs-Appliance-9.0.1.0.24960345.ova
The manifest does not validate
```

```

Opening VI target: vi://administrator%40vsphere.local@vcenter.lab.local:443/vcenter-dc01/host/vcenter-cl01/
Deploying to VI: vi://administrator%40vsphere.local@vcenter.lab.local:443/vcenter-dc01/host/vcenter-cl01/
Disk progress: 1%...2%...3%...
...
Disk progress: 98%...99%
Transfer Completed
Completed successfully

```

What each line means:

Message	Meaning
Opening OVA source	ovftool is reading the OVA file from your local disk
The manifest does not validate	Expected warning — we used <code>--skipManifestCheck</code>
Opening VI target	Connecting to vCenter and authenticating
Deploying to VI	Starting the upload to the target cluster
Disk progress: X%	The OVA disk images are being uploaded to the datastore
Transfer Completed	All disk images have been uploaded
Completed successfully	The VM has been created in vCenter

12.7 Troubleshooting ovftool Failures

Error	Cause	Fix
Locator does not refer to an object	Wrong datacenter or cluster name in the target URI	Verify the datacenter name with: <code>py -3 E:\VCF-Depot\get_env.py</code>
Error: No host is compatible with the virtual machine	Not enough host resources for the selected deployment size	Use <code>--deploymentOption="small"</code> instead of "medium"
HTTP Error 401	Wrong username or password in the URI	Check that <code>@</code> is <code>%40</code> and <code>!</code> is <code>%21</code> in the URL
Transfer failed at X%	Network timeout	Retry the same command — it will overwrite the partial VM
A virtual machine with the name 'logs' already exists	VM was not deleted	Delete the existing VM first (see Phase 6)

13. Phase 8 — Post-Deploy: Resize and Power On

After ovftool completes, the VM exists but is **powered off** with only 8 GB RAM (Small size). We need to increase RAM to 16 GB, enable hot-add, and power on.

13.1 Create the Configuration Script

Step 22: Create a file called `E:\VCF-Depot\configure_logs_vm.py` with this **complete script**:

```

import urllib.request
import json
import ssl

```

```

import base64
import time

# SSL and authentication
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE

password = "<your-password>"
credentials = "administrator@vsphere.local:" + password
encoded = base64.b64encode(credentials.encode()).decode()

request = urllib.request.Request(
    "https://vcenter.lab.local/api/session",
    data=b"",
    headers={"Authorization": "Basic " + encoded},
    method="POST"
)
response = urllib.request.urlopen(request, context=ctx)
session = json.loads(response.read())
headers = {
    "vmware-api-session-id": session,
    "Content-Type": "application/json"
}
print("vCenter session OK")

# Find the new logs VM by name
print("\nSearching for VM named 'logs'...")
req2 = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm",
    headers=headers
)
resp2 = urllib.request.urlopen(req2, context=ctx)
vms = json.loads(resp2.read())
logs_vm = None
for v in vms:
    if v["name"] == "logs":
        logs_vm = v["vm"]
        break

if not logs_vm:
    print("ERROR: No VM named 'logs' found in inventory!")
    print("Did the OVA deployment complete successfully?")
    exit(1)

print("Found: " + logs_vm)

# Show current config
req3 = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm/" + logs_vm,
    headers=headers
)
resp3 = urllib.request.urlopen(req3, context=ctx)
vm = json.loads(resp3.read())
print(" CPU: " + str(vm["cpu"]["count"]) + " vCPU")
print(" Memory: " + str(vm["memory"]["size_MiB"]) + " MiB")
print(" Power: " + vm["power_state"])

# Change 1: Increase RAM to 16 GB (16384 MiB)
print("\nSetting memory to 16384 MiB (16 GB)...")
data = json.dumps({"size_MiB": 16384}).encode()
req4 = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm/" + logs_vm
    + "/hardware/memory",
    data=data, headers=headers, method="PATCH"
)
try:
    resp4 = urllib.request.urlopen(req4, context=ctx)
    print(" Memory: " + str(resp4.status) + " OK")
except urllib.error.HTTPError as e:

```

```

        print(" Memory error: " + str(e.code) + " - "
              + e.read().decode()[:200])

# Change 2: Enable memory hot-add
print("Enabling memory hot-add...")
data = json.dumps({"hot_add_enabled": True}).encode()
req5 = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm/" + logs_vm
    + "/hardware/memory",
    data=data, headers=headers, method="PATCH"
)
try:
    resp5 = urllib.request.urlopen(req5, context=ctx)
    print(" Memory hot-add: " + str(resp5.status) + " OK")
except urllib.error.HTTPError as e:
    print(" Error: " + str(e.code) + " - "
          + e.read().decode()[:200])

# Change 3: Enable CPU hot-add
print("Enabling CPU hot-add...")
data = json.dumps({"hot_add_enabled": True}).encode()
req6 = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm/" + logs_vm
    + "/hardware/cpu",
    data=data, headers=headers, method="PATCH"
)
try:
    resp6 = urllib.request.urlopen(req6, context=ctx)
    print(" CPU hot-add: " + str(resp6.status) + " OK")
except urllib.error.HTTPError as e:
    print(" Error: " + str(e.code) + " - "
          + e.read().decode()[:200])

# Change 4: Power on
print("\nPowering on VM...")
req7 = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm/" + logs_vm
    + "/power?action=start",
    data=b"", headers=headers, method="POST"
)
try:
    resp7 = urllib.request.urlopen(req7, context=ctx)
    print(" Power on: " + str(resp7.status) + " OK")
except urllib.error.HTTPError as e:
    print(" Error: " + str(e.code) + " - "
          + e.read().decode()[:300])

# Verify final configuration
print("\nWaiting 10 seconds for boot...")
time.sleep(10)
req8 = urllib.request.Request(
    "https://vcenter.lab.local/api/vcenter/vm/" + logs_vm,
    headers=headers
)
resp8 = urllib.request.urlopen(req8, context=ctx)
vm2 = json.loads(resp8.read())
print("\nFinal VM Configuration:")
print(" VM ID: " + logs_vm)
print(" Name: " + str(vm2.get("name")))
print(" CPU: " + str(vm2["cpu"]["count"]) + " vCPU"
      + ", hot_add=" + str(vm2["cpu"]["hot_add_enabled"]))
print(" Memory: " + str(vm2["memory"]["size_MiB"]) + " MiB"
      + ", hot_add=" + str(vm2["memory"]["hot_add_enabled"]))
print(" Power State: " + vm2["power_state"])

```

13.2 Run the Configuration Script

Step 23: Run:

```
py -3 E:\VCF-Depot\configure_logs_vm.py
```

Expected output:

```
vCenter session OK

Searching for VM named 'logs'...
Found: vm-11016
  CPU:      4 vCPU
  Memory: 8192 MiB
  Power: POWERED_OFF

Setting memory to 16384 MiB (16 GB)...
  Memory: 204 OK
Enabling memory hot-add...
  Memory hot-add: 204 OK
Enabling CPU hot-add...
  CPU hot-add: 204 OK

Powering on VM...
  Power on: 204 OK

Waiting 10 seconds for boot...

Final VM Configuration:
  VM ID:      vm-11016
  Name:       logs
  CPU:        4 vCPU, hot_add=True
  Memory:     16384 MiB, hot_add=True
  Power State: POWERED_ON
```

VM configured and powered on. The new Logs appliance has 4 vCPU, 16 GB RAM, CPU/memory hot-add enabled, and is POWERED_ON.

14. Phase 9 — Verify the New Appliance

After powering on, the appliance needs **5-10 minutes** to:

1. Boot the Photon OS
2. Apply OVF properties (hostname, IP, DNS, gateway)
3. Format the `/storage/core` data partition (first boot only)
4. Start Cassandra
5. Start the Loginsight daemon
6. Start the Tomcat web server on ports 80 and 443

Step 24: Wait 5 minutes, then run the connectivity test script from Phase 2:

```
py -3 E:\VCF-Depot\test_connectivity.py
```

Expected output (after services are up):

```
Testing connectivity to 192.168.1.242...
=====
```

```
Port 22 (SSH): OPEN
Port 80 (HTTP): OPEN
Port 443 (HTTPS): OPEN
Port 514 (Syslog TCP): OPEN or REFUSED (normal if not configured)
Port 9000 (CFAPI): OPEN
Port 9543 (CFAPI over SSL): OPEN
```

```
Testing HTTPS request to web UI...
Status: 200
```

If ports 80 and 443 are still REFUSED after 10 minutes: The first boot initialization may be slow. The `/storage/core` partition (482 GB) takes time to format on first boot. Wait another 5 minutes and test again. You can also check the service status using the Guest Operations diagnostic script from Phase 3 (change `VM_ID` to the new VM ID).

15. Phase 10 — Initial Configuration Wizard

15.1 Open the Web UI

Step 25: Open a web browser on your workstation. Any modern browser works: - Google Chrome (recommended) - Microsoft Edge - Mozilla Firefox

Step 26: In the address bar, type the following URL and press **Enter**:

```
https://192.168.1.242
```

Step 27: You will see a **certificate warning** because the appliance uses a self-signed SSL certificate. This is normal for a new deployment.

- **In Chrome:** Click **Advanced** → Click **Proceed to 192.168.1.242 (unsafe)**
- **In Edge:** Click **Advanced** → Click **Continue to 192.168.1.242 (unsafe)**
- **In Firefox:** Click **Advanced...** → Click **Accept the Risk and Continue**

Step 28: The **VCF Operations for Logs Initial Configuration** wizard appears. It has 6 steps.

15.2 Wizard Step 1: Admin Password

Field	What to Enter
New Password	Enter a strong password (e.g., <code><your-password></code>)
Confirm Password	Re-enter the same password

This sets the `admin` user password for the web UI. This is **separate** from the root OS password.

Click **Save and Continue**.

15.3 Wizard Step 2: License Key

Option	What to Do
Enter a License Key	If you have a license key, enter it here
Skip	Click Skip to use the 60-day evaluation period

Click **Save and Continue** (or **Skip**).

15.4 Wizard Step 3: General Configuration

Field	Value	Notes
Hostname	logs.lab.local	Should already be pre-filled from OVF properties

Verify the hostname is correct and click **Save and Continue**.

15.5 Wizard Step 4: CEIP

Field	What to Do
Join the VMware Customer Experience Improvement Program	Uncheck this box in a lab environment

Click **Save and Continue**.

15.6 Wizard Step 5: Time Configuration (NTP)

Field	Value
NTP Servers	192.168.1.230

Type the NTP server address and click **Save and Continue**.

Why NTP matters: VCF Operations for Logs uses timestamps for all log events. If the appliance clock drifts, log timestamps will be wrong, and integration with VCF Operations may fail due to certificate time validation errors.

15.7 Wizard Step 6: SMTP

Field	What to Do
SMTP Server	Leave blank unless you need email alerts
Port	Leave default (25)

Click **Save and Continue** (or **Skip**).

15.8 Finish the Wizard

Click **Finish** (or **Save**). The appliance will:

1. Initialize the internal database

2. Create default content packs
3. Configure the web server
4. Redirect you to the login page (this takes 2-3 minutes)

Step 29: When the login page appears, log in with:

Field	Value
Username	admin
Password	The password you set in Step 1 of the wizard

Initial Configuration Complete. You should now see the VCF Operations for Logs dashboard. The appliance is functional and ready for integration.

16. Phase 11 — Integrate with VCF Operations

16.1 Configure on the Logs Side

Step 30: In your browser, navigate to:

`https://192.168.1.242`

Step 31: Log in as **admin** with the password you set during the wizard.

Step 32: Navigate to the integration settings:

1. Click the **gear icon** (Administration) in the top-right corner of the page
2. In the left sidebar, click **Configuration**
3. Click **VCF Operations Integration** (or **Aria Operations Integration**)

Step 33: Enter the VCF Operations connection details:

Field	Value
Host	192.168.1.77
User	admin
Password	(your VCF Operations admin password)

Step 34: Click **Test Connection**.

- If you see **"Connection Successful"**: Click **Save**.
- If you see a **certificate warning**: Click **Accept** to trust the VCF Operations certificate, then click **Test Connection** again.
- If the **test fails**: Verify that VCF Operations at 192.168.1.77 is running and reachable.

16.2 Configure on the VCF Operations Side

Step 35: Open a new browser tab and navigate to:

`https://192.168.1.77`

Step 36: Log in as **admin** with your VCF Operations admin password.

Step 37: Navigate to the Logs integration:

1. Click **Administration** in the left sidebar
2. Click **Solutions** (under Management)
3. Find **VMware VCF Operations for Logs** in the list
4. Click **Configure** (or the gear icon next to it)

Step 38: Enter the Logs connection details:

Field	Value
Host	192.168.1.242
User	admin
Password	(your Logs admin password from the wizard)

Step 39: Click **Test**. If a certificate prompt appears, click **Accept Certificate**.

Step 40: Click **Save**.

16.3 Verify the Integration

On VCF Operations for Logs (192.168.1.242):

1. Navigate to **Administration** → **Hosts**
2. Verify the local node shows status **Running**
3. Navigate to **Dashboards** — default dashboards should be visible

On VCF Operations (192.168.1.77):

1. Navigate to **Administration** → **Solutions**
2. Verify **VCF Operations for Logs** shows status **Connected**
3. Navigate to any VM in the inventory → click the **Logs** tab
4. Click **Launch in Context** — this should open the Logs UI showing logs for that VM

Integration Complete. VCF Operations and VCF Operations for Logs are now connected. Log data will flow from ESXi hosts and vCenter into the Logs appliance, and you can access logs directly from VCF Operations.

17. Summary of All Root Causes

#	Root Cause	Impact	Resolution
1	VM deployed as Extra Small (2 vCPU / 8 GB RAM)	Insufficient memory for Loginsight (4 GB) + Cassandra (3 GB) + OS	Redeployed as Small + resized to 16 GB
2	Cassandra crash loop	Port 9042 never stayed up; all services depend on Cassandra	Resolved by fresh deployment
3	Missing <code>jnr-posix</code> Java library (<code>NoClassDefFoundError</code>)	Cassandra cannot initialize native POSIX operations	Corrupted install; resolved by fresh OVA
4	Stuck upgrade state (<code>UpgradeService</code> error loop)	Application restarts endlessly	Resolved by fresh deployment

18. Lessons Learned

1. **Always deploy VCF Operations for Logs with at least Small sizing (4 vCPU / 8 GB) and increase RAM to 16 GB.** The Extra Small option is documented as "test/POC only" and should never be used. Even Small (8 GB) is marginal.
2. **The vCenter Guest Operations API is invaluable when SSH is unavailable or the application is down.** All diagnostics were performed remotely through `POST /api/vcenter/vm/{id}/guest/processes?action=create`. This should be a standard tool in any VMware administrator's toolkit.
3. **Check the runtime log at `/storage/var/loginsight/runtime.log` first.** This is the single most informative log file on the appliance.
4. **An empty `/storage/core` partition means the appliance was never initialized.** If no data exists, there is nothing to lose by redeploying fresh.
5. **ovftool is the fastest way to deploy OVAs programmatically.** All OVF properties can be set at deployment time, eliminating manual VAMI configuration.
6. **When Medium or Large deployments fail with "No host is compatible",** deploy as Small first and resize the VM before powering on.
7. **Always enable CPU and memory hot-add** on deployed appliances for future resizing without downtime.

Appendix A — Complete ovftool Deployment Command

Copy and paste this command into a Git Bash terminal. Adjust the values in the parameter table in Section 12.4 before running.

```
# Workstation-side: identical to Section 12.5 – full ovftool deploy command for the Logs appliance.
"/c/Program Files (x86)/VMware/VMware Workstation/OVFTool/ovftool.exe" \
--acceptAllEulas \
--skipManifestCheck \
--noSSLVerify \
--X:injectOvfEnv \
--X:enableHiddenProperties \
--name="logs" \
--deploymentOption="small" \
--diskMode="thin" \
--datastore="vcenter-cl01-ds-vsan01" \
```

```
--network="vcenter-cl01-vds01-pg-esx-mgmt" \
--prop:rootpw="<your-password>" \
--prop:hostname="logs.lab.local" \
--prop:preferipv6="False" \
--prop:fips="False" \
--prop:vami.gateway.VMware_vCenter_Log_Insight="192.168.1.1" \
--prop:vami.domain.VMware_vCenter_Log_Insight="lab.local" \
--prop:vami.searchpath.VMware_vCenter_Log_Insight="lab.local" \
--prop:vami.DNS.VMware_vCenter_Log_Insight="192.168.1.230" \
--prop:vami.ip0.VMware_vCenter_Log_Insight="192.168.1.242" \
--prop:vami.netmask0.VMware_vCenter_Log_Insight="255.255.255.0" \
"E:/VCF-Depot/PROD/COMP/VRLI/Operations-Logs-Appliance-9.0.1.0.24960345.ova" \
"vi://administrator%40vsphere.local:<url-encoded-password>@vcenter.lab.local/vcenter-dc01/host/vcenter-cl01/"
```

```
# PowerShell 5.1 sibling – Windows-native paths, backtick line continuation, SINGLE-quoted values
# (so '!' and '$' are not expanded). URL-encode '@' as %40 in the vi:// URI exactly as in bash.

& 'C:\Program Files (x86)\VMware\VMware Workstation\OVFTool\ovftool.exe' `
--acceptAllEulas `
--skipManifestCheck `
--noSSLVerify `
--X:injectOvfEnv `
--X:enableHiddenProperties `
--name=logs `
--deploymentOption=small `
--diskMode=thin `
'--datastore=vcenter-cl01-ds-vsan01' `
'--network=vcenter-cl01-vds01-pg-esx-mgmt' `
'--prop:rootpw=<your-password>' `
'--prop:hostname=logs.lab.local' `
'--prop:preferipv6=False' `
'--prop:fips=False' `
'--prop:vami.gateway.VMware_vCenter_Log_Insight=192.168.1.1' `
'--prop:vami.domain.VMware_vCenter_Log_Insight=lab.local' `
'--prop:vami.searchpath.VMware_vCenter_Log_Insight=lab.local' `
'--prop:vami.DNS.VMware_vCenter_Log_Insight=192.168.1.230' `
'--prop:vami.ip0.VMware_vCenter_Log_Insight=192.168.1.242' `
'--prop:vami.netmask0.VMware_vCenter_Log_Insight=255.255.255.0' `
'E:/VCF-Depot/PROD/COMP/VRLI/Operations-Logs-Appliance-9.0.1.0.24960345.ova' `
"vi://administrator%40vsphere.local:<url-encoded-password>@vcenter.lab.local/vcenter-dc01/host/vcenter-cl01/"
```

Appendix B — VM Discovery Script

Save as `find_logs_vm.py`. See Section 6.1 for the complete script and usage instructions.

Appendix C — Guest Operations Diagnostic Script

Save as `diag_logs.py`. See Section 8.3 for the complete script and usage instructions.

Appendix D — VM Resize and Power-On Script

Save as `configure_logs_vm.py`. See Section 13.1 for the complete script and usage instructions.

Appendix E — VM Deletion Script

Save as `delete_logs_vm.py` . See Section 11.2 for the complete script and usage instructions.

Index

Term	Section
ActiveMQ	8.5, 9.4
Callback mechanism	8.2
Cassandra	8.5, 9.2, 9.3, 10.1
Cassandra crash loop	9.2
Cassandra port 9042	7.2, 8.5, 9.2, 10.1
Certificate warning (browser)	15.1
CPU hot-add	10.1, 13.1
DRS placement failure	12.3, 12.7
Extra Small deployment	9.1, 12.3
Guest Operations API	8.1, 8.2, 8.3
Hot-add (CPU/memory)	10.1, 13.1, 18
Initial Configuration Wizard	15
Java heap (<code>-Xmx3968m</code>)	9.1
<code>jnr/posix/POSIXHandler</code>	9.3
Loginsight service	8.5, 10.1
Memory hot-add	10.1, 13.1
<code>NoClassDefFoundError</code>	9.3
NTP configuration	15.6
OVA deployment	12
OVA file location	12.1
OVF properties	12.2, 12.4
ovftool	3.2, 12.4, 12.5, Appendix A
ovftool flags	12.4
ovftool target URI	12.4
Photon OS	6.2
Python installation	3.1
Runtime log	8.5, 9.3, 9.4

Term	Section
/storage/core	8.5, 11.1
/storage/var/loginsight/runtime.log	8.5
ss -tlnp	8.5, 10.1
systemctl	8.5
UpgradeService	9.4
URL encoding (%40 , %21)	12.4
vCenter REST API	4
vCenter REST API session	4.1, 4.2
VCF Operations integration	16
VM deletion	11
VM resize	10.1, 13.1
VM sizing requirements	9.1, 12.3

Revision History

Version	Date	Author	Notes
2.0	March 25, 2026	Virtual Control LLC	Initial troubleshooting/redeployment/integration handbook publication.
2.1	2026-05-08	Virtual Control LLC	Option B treatment: added Windows PowerShell 5.1 sibling blocks alongside the two workstation-side bash blocks (Section 12.5 ovftool deploy and Appendix A repeat). Added shell-translation callout linking to ../POWERSHELL-TRANSLATION-REFERENCE.md . Added context comments to bash blocks. No content restructuring; binder cover preserved. Appliance-side commands (Photon systemctl , ss , tail , log paths) intentionally left bash-only — they run inside the Logs VM.

This document was prepared by Virtual Control LLC. All commands, outputs, and procedures reflect the actual troubleshooting performed on March 24-25, 2026, in the VCF 9 lab environment.

© 2026 Virtual Control LLC. All rights reserved.